

Experiment 1

Aim: To explore and implement advanced SQL techniques, including joins, subqueries, aggregate functions, conditional statements, and date operations for efficient database management and data retrieval.

Code:

Problem 1: Write an SQL query to determine the number of customer accounts in each income category. The categories are defined as:

- Low Income: Income less than 20,000
- Medium Income: Income between 20,000 and 50,000 (inclusive)
- High Income: Income greater than 50,000

Ensure that all three categories are displayed even if no accounts belong to a particular category.

Solution:

```
SELECT 'LOW SALARY' AS CATEGORY,  
       COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME < 20000
```

UNION ALL

```
SELECT 'AVERAGE SALARY' AS CATEGORY,  
       COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME >= 20000  
       AND INCOME <= 50000
```

UNION ALL

```
SELECT 'HIGH SALARY' AS CATEGORY,  
       COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME > 50000;
```

Problem 2:

Develop a Company Employee Database by performing the following operations:

1. Create **Departments**, **Employees**, and **Employee_Salary** tables.
2. Insert suitable sample records into each table.
3. Retrieve employee details along with their department and salary using SQL JOIN.
4. Increase the salary of employees working in the **Finance** department by **15%**.
5. Remove employee salary records where the salary is below **₹35,000**.
6. Display employees whose salary is greater than the average salary of all employees.

Solution:

-- Create Tables

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);
```

```
CREATE TABLE Salaries (  
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,  
    salary INT NOT NULL  
);
```

-- Insert Sample Records

```
INSERT INTO Departments (dept_id, dept_name) VALUES  
(10, 'HR'),  
(20, 'IT'),  
(30, 'Sales');
```

```
INSERT INTO Employees (emp_id, name, dept_id) VALUES  
(1, 'Alice', 10),  
(2, 'Bob', 10),  
(3, 'Charlie', 20),  
(4, 'David', 20),  
(5, 'Emma', 30);
```

```
INSERT INTO Salaries (emp_id, salary) VALUES  
(1, 50000),  
(2, 25000),  
(3, 80000),  
(4, 40000),  
(5, 45000);
```

-- Display Employee Details Using JOIN

```
SELECT E.*, D.*, S.*  
FROM Employees AS E  
JOIN Departments AS D  
ON E.dept_id = D.dept_id  
JOIN Salaries AS S  
ON E.emp_id = S.emp_id;
```

-- Increase HR Employee Salary by 10%

```
UPDATE Salaries  
SET salary = salary + salary * 0.10  
WHERE emp_id IN  
(
```

```

SELECT E.emp_id
FROM Employees AS E
JOIN Departments AS D
ON E.dept_id = D.dept_id
WHERE D.dept_name = 'HR'
);

-- Display Updated Employee Details
SELECT E.*, D.*, S.*
FROM Employees AS E
JOIN Departments AS D
ON E.dept_id = D.dept_id
JOIN Salaries AS S
ON E.emp_id = S.emp_id;

-- Delete Salary Records Below 30000
DELETE FROM Salaries
WHERE salary < 30000;

-- Display Employees Earning More Than Company Average Salary
SELECT E.emp_id
FROM Employees AS E
JOIN Salaries AS S
ON E.emp_id = S.emp_id
WHERE S.salary >
(
    SELECT AVG(salary)
    FROM Salaries
);

```

Problem 3: Create a table named **pizza_toppings** to store different pizza toppings and their ingredient costs. Insert the given sample records into the table. Then perform the following operations:

1. Display all available pizza toppings.
2. Display toppings whose ingredient cost is greater than **0.50**.
3. Display all toppings in ascending order of ingredient cost.
4. Find the topping with the highest ingredient cost.

Solution:

```

-- Create Table
CREATE TABLE pizza_toppings (
    topping_name VARCHAR(50) PRIMARY KEY,
    ingredient_cost NUMERIC(4,2) NOT NULL
);

-- Insert Sample Records
INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
('Pepperoni', 0.50),
('Sausage', 0.70),

```

```

('Chicken', 0.55),
('Onions', 0.25),
('Extra Cheese', 0.40);

-- Display All Records
SELECT *
FROM pizza_toppings;

-- Display Toppings Costing More Than 0.50
SELECT *
FROM pizza_toppings
WHERE ingredient_cost > 0.50;

-- Display Toppings in Ascending Order of Cost
SELECT *
FROM pizza_toppings
ORDER BY ingredient_cost ASC;

-- Find the Most Expensive Topping
SELECT *
FROM pizza_toppings
WHERE ingredient_cost = (
    SELECT MAX(ingredient_cost)
    FROM pizza_toppings
);

```

Learning Outcome:

1. Understand and implement advanced SQL queries for effective data retrieval.
2. Create and manage relational database tables using SQL DDL commands.
3. Insert, update, delete, and retrieve data using SQL DML statements.
4. Apply SQL joins to combine data from multiple related tables.